

Lecture Title: Runtime Library and Environment Variables in OpenMP

Subject: High Performance Computing (HPC)

Target Audience: UG/PG Computer Science or IT Students

Lecture Duration: 60 Minutes

Tools: GCC with OpenMP (-fopenmp), Terminal, Code Editor

Learning Objectives

By the end of this lecture, students will:

1. Understand the purpose of OpenMP runtime library functions.
 2. Use key OpenMP routines like `omp_get_thread_num()` and `omp_get_num_threads()`.
 3. Modify program behavior using OpenMP environment variables (e.g., `OMP_NUM_THREADS`).
 4. Practice with live code demonstrations and environment control.
-

1. Introduction to OpenMP Runtime Library

OpenMP provides a **runtime library** that includes functions to:

- Query thread IDs and counts
- Manage thread affinity and nesting
- Measure execution time
- Set or get scheduling policies

Key Header File:

```
#include <omp.h>
```

2. Important Runtime Functions

Function	Description
<code>omp_get_thread_num()</code>	Returns the thread number (ID) of calling thread

omp_get_num_threads()	Returns total number of threads in current region
omp_get_max_threads()	Max threads the system may use in parallel
omp_set_num_threads(int)	Sets the number of threads for next parallel region
omp_get_wtime()	Returns wall-clock time

✓ Example 1: Basic Runtime Query

```
#include <stdio.h>

#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int total = omp_get_num_threads();
        printf("Thread %d of %d is running\n", id, total);
    }
    return 0;
}
```

🔍 Expected Output (varies depending on thread count):

```
Thread 0 of 4 is running
Thread 1 of 4 is running
...
```

📁 3. Environment Variables in OpenMP

Environment variables allow you to **control** the execution of an OpenMP program **without changing the source code**.

📖 Common OpenMP Environment Variables

Variable	Function
OMP_NUM_THREADS	Sets number of threads
OMP_SCHEDULE	Controls loop scheduling type (static/dynamic)
OMP_DYNAMIC	Enables dynamic thread adjustment
OMP_PROC_BIND	Controls thread-to-core binding

✅ Example 2: Controlling Threads Using OMP_NUM_THREADS

Code:

```
#include <stdio.h>

#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Thread ID: %d\n", omp_get_thread_num());
    }
    return 0;
}
```

Terminal:

```
export OMP_NUM_THREADS=6

./a.out
```

🔍 **Effect:** Program uses 6 threads instead of default.

💡 4. Using omp_set_num_threads() (Programmatic Control)

```
#include <stdio.h>

#include <omp.h>
```

```

int main() {
    omp_set_num_threads(3); // Set number of threads programmatically

    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }

    return 0;
}

```

 This overrides the OMP_NUM_THREADS environment variable **for this region only**.

5. Measuring Time with omp_get_wtime()

```

#include <stdio.h>
#include <omp.h>

int main() {
    double start, end;
    start = omp_get_wtime();

    #pragma omp parallel for
    for(int i = 0; i < 100000000; i++); // Dummy workload

    end = omp_get_wtime();
    printf("Execution Time: %f seconds\n", end - start);
    return 0;
}

```

📌 **Use Case:** Performance benchmarking of parallel regions.

🖥️ Live Demo Activities

1. Run a program with default threads.
 2. Use `export OMP_NUM_THREADS=8` and rerun.
 3. Compare wall-clock time of serial vs parallel loop with `omp_get_wtime()`.
 4. Change `OMP_SCHEDULE=dynamic` and observe loop iteration behavior.
-

🗨️ Discussion Prompt for Students

- What happens if `OMP_NUM_THREADS` is more than your CPU's core count?
 - What's the difference between using `omp_set_num_threads()` and the environment variable?
 - What are potential downsides of dynamic thread allocation?
-

✅ Summary

- **Runtime library functions** give control and insight into thread behavior.
 - **Environment variables** allow flexible, code-independent configuration.
 - `omp_get_thread_num()` and `omp_get_num_threads()` are foundational for understanding thread structure.
 - Tools like `omp_get_wtime()` help in **performance measurement**.
-

🎓 Hands-On Assignment

Write a C/OpenMP program that:

- Accepts matrix size `N` as input.
- Uses `OMP_NUM_THREADS` to set thread count.
- Computes the sum of all elements of an `NxN` matrix using `omp parallel for`.
- Measures and prints time taken.

